Lecture 12 — Stacks, Queues, Deques, Monotonic Stack & Monotonic Queue

> Year Curriculum · Prerequisites: Week 2 (STL), Week 6 (two pointers & sliding window), Week 7 (prefix sums) · Estimated time: 2 × 90 min

1. Overview

Stacks and queues are the simplest data structures beyond arrays: you may only touch the ends. That restriction is exactly what makes them powerful. A stack remembers "the most recent unfinished thing", which is the structure of nesting (brackets, recursion, DFS). A queue remembers "who arrived first", which is the structure of simulations and BFS. A deque does both.

The real competitive-programming payoff is the **monotonic** versions. By throwing away elements that can never matter again, a stack answers "nearest smaller element to the left" for every position in O(n) total, and a deque maintains the minimum of a sliding window in O(n) total. These two patterns solve a surprising number of problems: histogram rectangles, stock spans, contribution counting, and later in the course, DP optimisations (Regionals: monotonic-queue DP) and the Cartesian tree.

Goals for this lecture:

- use `stack`, `queue`, `deque` (and `vector` as a stack) fluently and safely;
- solve matching/nesting problems with a stack;
- derive and code the four monotonic-stack variants (previous/next smaller/greater);
- solve the largest-rectangle-in-histogram problem;
- maintain a sliding-window minimum or maximum with a monotonic deque;
- justify O(n) running time with amortized analysis.

2. The containers

Structure	Discipline	STL	Key operations (all O(1))
Stack	last in, first out	`stack<T>` or `vector<T>`	`push`/`push_back`, `top`/`back`, `pop`/`pop_back`
Queue	first in, first out	`queue<T>`	`push`, `front`, `pop`, `back`

Structure	Discipline	STL	Key operations (all O(1))
Deque	both ends	`deque<T>`	`push_front`, `push_back`, `pop_front`, `pop_back`, `front`, `back`, `[i]`

Practical notes:

- `stack::pop()` and `queue::pop()` return `void`. Read `top()`/`front()` before popping.
- Calling `top()`, `front()` or `pop()` on an empty container is undefined behaviour; it may "work" locally and crash on the judge. Always guard with `!st.empty()`.
- A `vector` used as a stack is slightly faster than `std::stack` and lets you iterate over or index into the stack, which is often handy when debugging monotonic stacks. `std::stack` and `std::queue` are adapters on top of `deque` by default.
- `deque` supports random access but its memory is not contiguous; for a pure stack prefer `vector`.

3. Stacks for nesting

3.1 Bracket matching

A string over `()[]{}` is balanced if every closer matches the most recent unmatched opener of the same type. Push openers; on a closer, the stack must be non-empty and its top must be the matching opener; at the end the stack must be empty. Each character is pushed and popped at most once, so the check is O(n). Three distinct failure modes appear: a closer with an empty stack (`)(`), a type mismatch (`([)]`), and leftover openers (`((`).

With only one bracket type the stack degenerates into a counter (the depth): +1 for `(`, −1 for `)`. The string is balanced iff the counter never goes negative and ends at 0. This "prefix balance" view connects to Week 7 prefix sums and to Catalan numbers (Week 11).

Worked example — Codeforces 5C *Longest Regular Bracket Sequence*. Push indices instead of characters, and start with a sentinel index −1 on the stack. On `(`, push its index. On `)`, pop; if the stack becomes empty, push the current index as the new sentinel (an unmatched closer is a barrier); otherwise the substring from `top + 1` to the current index is balanced, so update the best length with `i - top`. Storing **indices** rather than values is the recurring trick: indices let you compute lengths and distances.

3.2 Other nesting structures

The call stack *is* a stack, so every recursive algorithm can be made iterative with an explicit stack (useful when recursion depth would overflow, see Week 8). Expression evaluation (operators and operands on two stacks), undo histories, and iterative DFS (Week 16) are further examples.

4. Queues

A queue processes items in arrival order. Uses include direct simulations (customers at a counter, round-robin scheduling, Josephus-style elimination with small n) and, most importantly, breadth-first search in Week 16, where the queue guarantees vertices are processed in non-decreasing distance. A queue is also a natural container for "the last k items" of a stream: push the new item, pop the front when the size exceeds k.

5. The monotonic stack

5.1 Derivation

Problem (CSES 1645 *Nearest Smaller Values*): for every i, find the largest $j < i$ with $a[j] < a[i]$, or report 0.

Scan from left to right and ask: among the positions already seen, which ones could still be the answer for some *future* i? Suppose $j < k \le$ current position and $a[j] \ge a[k]$. Any future i that would accept j (because $a[j] < a[i]$) also accepts k (since $a[k] \le a[j] < a[i]$), and k is closer. So j can never be an answer again and can be discarded.

After discarding all such dominated positions, the surviving candidates have **strictly increasing values from left to right**: a stack whose values increase from bottom to top. To process a new i:

1. pop while the top's value is $\ge a[i]$ (those are now dominated by i, *and* none of them is a valid answer for i since they are not smaller);
2. the top, if any, is the nearest smaller value to the left of i;
3. push i.

Trace for a = 2 5 1 4 8 3 2 5 (1-indexed):

i	a[i]	popped	answer	stack after (index:value)
1	2	—	0	1:2
2	5	—	1	1:2, 2:5
3	1	2, 1	0	3:1
4	4	—	3	3:1, 4:4
5	8	—	4	3:1, 4:4, 5:8
6	3	5, 4	3	3:1, 6:3
7	2	6	3	3:1, 7:2
8	5	—	7	3:1, 7:2, 8:5

5.2 Amortized analysis

A single step can pop many elements (step 3 pops two), so the worst case per step is O(n). But every index is pushed exactly once and popped at most once, so across the whole scan there are at most 2n push/pop operations. The total is O(n), i.e. **amortized O(1) per element**. This "charge each operation to the element it moves" argument is the standard way to analyse monotonic structures and two-pointer loops (Week 6).

5.3 Variants

Goal	Scan direction	Pop while top's value is
previous smaller	left to right	≥ a[i]
previous smaller-or-equal	left to right	> a[i]
previous greater	left to right	≤ a[i]
next smaller	right to left	≥ a[i]
next greater	right to left	≤ a[i]

Alternatively, "next smaller" can be computed in the same left-to-right pass: when index i pops index j, i is the first element after j that is ≤ a[j] (so with the `>=` condition you get "next smaller or equal"; use `>` to get strictly smaller). Choosing between strict and non-strict comparisons matters whenever values repeat; decide deliberately, and test with equal values.

5.4 Largest rectangle in a histogram

Problem (CSES 1142 *Advertisement*): given bar heights h[0..n−1] of width 1, find the largest rectangle inside the histogram.

Consider an optimal rectangle. Its height is limited by the lowest bar it covers, and if it were lower than that bar we could raise it, so its height equals h[i] for some bar i inside it. Conversely, for each bar i, the widest rectangle of height h[i] containing i extends left until the first bar lower than h[i] and right until the first bar lower than h[i]. With L[i] = index of the previous smaller bar (or −1) and R[i] = index of the next smaller bar (or n),

area(i) = h[i] · (R[i] − L[i] − 1),

and the answer is the maximum over i. Two monotonic-stack passes give L and R in O(n).

Worked example, h = 4 1 5 3 3 2 4 1:

i	0	1	2	3	4	5	6	7
h	4	1	5	3	3	2	4	1
L	−1	−1	1	1	1	1	5	−1

i	0	1	2	3	4	5	6	7
R	1	8	3	5	5	7	7	8
area	4	8	5	9	9	10	4	8

The answer is 10: height 2 over bars 2..6. Equal heights (bars 3 and 4) are not a problem: with strict "smaller" on both sides, each of them spans the full run, and both report the same correct area.

This problem generalises to the **maximal all-ones rectangle in a binary grid** (CSES 1147 *Maximum Building I*): for each row, compute for every column the number of consecutive free cells ending at this row, and run the histogram algorithm on that row — O(nm) overall.

5.5 Contribution counting

Many problems ask for a sum over all subarrays, such as Σ min(a[l..r]). Instead of enumerating $O(n^2)$ subarrays, ask for each i: in how many subarrays is a[i] the minimum? Those subarrays start after the previous smaller element and end before the next smaller element, giving $(i - L[i]) \cdot (R[i] - i)$ of them. With repeated values you must break ties consistently — "strictly smaller" on one side and "smaller or equal" on the other — so that each subarray's minimum is attributed to exactly one index. The total is O(n).

6. The monotonic deque

6.1 Sliding window minimum

Problem: for an array of n numbers and window length k, output min(a[i−k+1..i]) for each $i \geq k - 1$. Recomputing each window is O(nk); keeping a `multiset` of the window (Week 13) is O(n log k). A deque gives O(n).

Use the same domination argument as before, but now elements also **expire**. If $j < k$ are both in the window and $a[j] \geq a[k]$, then j is never the window minimum again: it leaves the window before k does and is not smaller. So the deque holds indices with **increasing values** from front to back, and the front is the current minimum. For each new index i:

1. pop from the back while a[back] ≥ a[i];
2. push i at the back;
3. if the front index is $\leq i - k$ it has left the window: pop it from the front;
4. if $i \geq k - 1$, report a[front].

Trace with a = 1 3 −1 −3 5 3 6 7 and k = 3 (deque shown as values):

i	a[i]	deque after	window min
0	1	1	—
1	3	1 3	—
2	−1	−1	−1
3	−3	−3	−3
4	5	−3 5	−3
5	3	−3 3	−3
6	6	3 6	3 (−3 at index 3 expired)
7	7	3 6 7	3

Each index is pushed once and popped at most once (from one end or the other), so the total is O(n). For a sliding maximum, flip the comparison.

Step 3 only needs to pop one element because at most one index expires per step; it is also fine to write it as a `while`. The algorithm also works for windows whose left end moves irregularly (as in two-pointer problems from Week 6), as long as both ends only move forward: push on the right, pop expired indices on the left.

6.2 Queue with aggregate via two stacks

The monotonic deque only works for min/max. For other associative operations (gcd, matrix product, …) use a queue built from two stacks: each stack stores pairs (value, aggregate of everything below and including it). Push onto the "in" stack; pop from the "out" stack, and when "out" is empty, move all of "in" into "out" (recomputing aggregates). The aggregate of the whole queue combines the tops of both stacks. Every element moves between stacks at most once, so all operations are amortized O(1). Combined with two pointers, this finds, for example, the shortest subarray whose gcd is 1 in O(n log A).

7. Reference implementations

7.1 Balanced brackets

```cpp
#include <bits/stdc++.h>
using namespace std;
// Is a bracket string over ()[]{} balanced?
bool balanced(const string& s) {
    stack<char> st;
    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') { st.push(c); continue; }
        if (st.empty()) return false;
        char o = st.top(); st.pop();
        if ((c == ')' && o != '(') || (c == ']' && o != '[') || (c == '}' && o != '{'))
```

```
            return false;
    }
    return st.empty();
}
int main() {
    string s;
    while (cin >> s) cout << s << " -> " << (balanced(s) ? "YES" : "NO") << "\n";
}
```

7.2 CSES 1645 — Nearest Smaller Values

```
#include <bits/stdc++.h>
using namespace std;
// CSES 1645 - Nearest Smaller Values: for each i, the closest j < i with a[j] < a[i]
// (1-indexed position), or 0 if none.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<int> a(n + 1);
    for (int i = 1; i <= n; i++) cin >> a[i];
    vector<int> st;                          // indices; a[st] strictly increasing bottom-
>top
    for (int i = 1; i <= n; i++) {
        while (!st.empty() && a[st.back()] >= a[i]) st.pop_back();
        cout << (st.empty() ? 0 : st.back()) << " \n"[i == n];
        st.push_back(i);
    }
}
```

7.3 CSES 1142 — Advertisement (largest rectangle in a histogram)

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
// CSES 1142 - Advertisement: largest rectangle in a histogram.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n; cin >> n;
    vector<ll> h(n);
    for (auto& x : h) cin >> x;
    vector<int> L(n), R(n), st;
    // L[i] = index of nearest bar to the left that is strictly lower (or -1)
    for (int i = 0; i < n; i++) {
        while (!st.empty() && h[st.back()] >= h[i]) st.pop_back();
        L[i] = st.empty() ? -1 : st.back();
        st.push_back(i);
    }
    st.clear();
    // R[i] = index of nearest bar to the right that is strictly lower (or n)
    for (int i = n - 1; i >= 0; i--) {
        while (!st.empty() && h[st.back()] >= h[i]) st.pop_back();
```

```
        R[i] = st.empty() ? n : st.back();
        st.push_back(i);
    }
    ll best = 0;
    for (int i = 0; i < n; i++) best = max(best, h[i] * (R[i] - L[i] - 1));
    cout << best << "\n";
}
```

Heights go up to 10^9 and widths up to $2 \cdot 10^5$, so the area needs `long long`.

7.4 Sliding window minimum

```
#include <bits/stdc++.h>
using namespace std;
// Sliding window minimum: print min of every window of length k.
int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, k; cin >> n >> k;
    vector<int> a(n);
    for (auto& x : a) cin >> x;
    deque<int> dq;                          // indices; a[dq] increasing front->back
    for (int i = 0; i < n; i++) {
        while (!dq.empty() && a[dq.back()] >= a[i]) dq.pop_back();  // dominated
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();                    // expired
        if (i >= k - 1) cout << a[dq.front()] << " \n"[i == n - 1];
    }
}
```

Input `8 3 / 1 3 -1 -3 5 3 6 7` prints `-1 -3 -3 -3 3 3`. (CSES 3221 generates its large input from a formula and asks for an aggregate of the window minimums rather than printing them all; read the statement carefully — the core deque loop is identical.)

8. Common mistakes

- **Empty-container access**: `st.top()` with no guard. Every `while` must check `!st.empty()` first, and the order in `&&` matters.
- **Values instead of indices**: if you need distances, widths or expiry, store indices and look up values through the array.
- **Strict vs non-strict comparison**: `>=` vs `>` changes the answer when values repeat. Decide what "smaller" means in the statement and test with duplicates. In contribution counting, inconsistent tie-breaking double-counts or misses subarrays.
- **Forgetting expiry** in the sliding window, or expiring with the wrong bound (`< i - k` instead of `<= i - k`), which yields windows of length k + 1.
- **Overflow**: histogram areas and subarray-min sums exceed 32 bits easily.
- **Clearing state** between the two passes or between test cases (`st.clear()`).

- **Popping from the wrong end** of the deque: dominated elements go from the back, expired ones from the front.

9. Practice problems

Warm-up:

1. **LeetCode 20 — Valid Parentheses** — https://leetcode.com/problems/valid-parentheses/
2. **Codeforces 343B — Alternating Current** — https://codeforces.com/problemset/problem/343/B (cancel adjacent equal symbols with a stack).
3. **Kattis evenup — Even Up Solitaire** — https://open.kattis.com/problems/evenup
4. **LeetCode 739 — Daily Temperatures** — https://leetcode.com/problems/daily-temperatures/ (next greater element).

Core:

5. **CSES 1645 — Nearest Smaller Values** — https://cses.fi/problemset/task/1645
6. **CSES 1142 — Advertisement** — https://cses.fi/problemset/task/1142
7. **CSES 3221 — Sliding Window Minimum** — https://cses.fi/problemset/task/3221
8. **Codeforces 5C — Longest Regular Bracket Sequence** — https://codeforces.com/problemset/problem/5/C
9. **Codeforces 547B — Mike and Feet** — https://codeforces.com/problemset/problem/547/B (each element as the minimum of its maximal window).

Challenge:

10. **CSES 1147 — Maximum Building I** — https://cses.fi/problemset/task/1147 (histogram per row).
11. **Codeforces 1407D — Discrete Centrifugal Jumps** — https://codeforces.com/problemset/problem/1407/D (DP with two monotonic stacks).
12. **LeetCode 907 — Sum of Subarray Minimums** — https://leetcode.com/problems/sum-of-subarray-minimums/ (contribution technique with tie-breaking).

10. Further reading

- Antti Laaksonen, *Competitive Programmer's Handbook*, Chapter 8 "Amortized analysis" (nearest smaller elements, sliding window minimum): https://cses.fi/book/book.pdf
- CP-Algorithms, "Minimum stack / Minimum queue": https://cp-algorithms.com/data_structures/stack_queue_modification.html
- USACO Guide, Gold "Stacks" and "Sliding Window": https://usaco.guide/gold/stacks , https://usaco.guide/gold/sliding-window

- Coming up: Week 13 (heaps and ordered sets — the O(log n) alternatives), Week 16 (queues in BFS), Regionals "DP patterns" (monotonic-queue DP optimisation).